

C 语言程序设计

第七讲 函数与模块化 作用范围和存储类别



张 华

作用范围和存储类别

- 标识符的作用范围
- 变量的存储类别
- 包含多个源文件的程序
 - ✿ 编译和链接
 - ✿ 外部函数
 - ✿ 内部函数

简介

■ 变量定义的完整格式：

存储类别 数据类型 变量名

✿ 数据类型

- 决定了变量占用存储空间的大小
- 取值范围

✿ 存储类别

- 决定了变量在内存中持续的时间（生存期）
- 在硬件中存放的位置

■ 其它属性

✿ 作用范围

- 变量可以被引用的程序部分（可见性）

作用范围

■ 程序中的所有标识符构成了名字空间。

✿ 需要一个机制来避免和解决名字的重名冲突问题。

■ 标识符的作用范围

✿ 能够引用该标识符的程序部分。

✿ 四种作用范围：

- 文件作用范围
- 函数作用范围
- 程序块作用范围
- 函数原型作用范围

作用范围

文件作用范围

- 在函数外声明的标识符，可以在所有函数中被引用。
- 包括：全局变量（在函数外定义的变量），函数。

```
int total;

int max(int, int);

void main() {
    ...
}

int limit;

int max(int x, int y) {
    ...
}
```

作用范围

从声明的位置
开始，到文件
的末尾。

作用范围

函数作用范围

- 在函数体内定义的标识符，只能在函数体内被引用。
- 包括：语句标号（无需单独声明）

```
void main() {  
    ...  
    goto loop;  
    ...  
    loop:...;  
}
```

作用范围

程序块作用范围

- 在程序块内声明的变量，在程序块内被引用。
- 包括：程序块内定义的变量（局部变量），函数的参数。

```
int max(int x, int y) {  
    ...  
}  
  
void main() {  
    int a;  
    ...  
    { int a;  
        ...  
    }  
    ...  
}
```

作用范围

从声明的位置
开始，到程序
块的右大括号

覆盖

同名变量，内部变量
“覆盖了”外部变量

作用范围

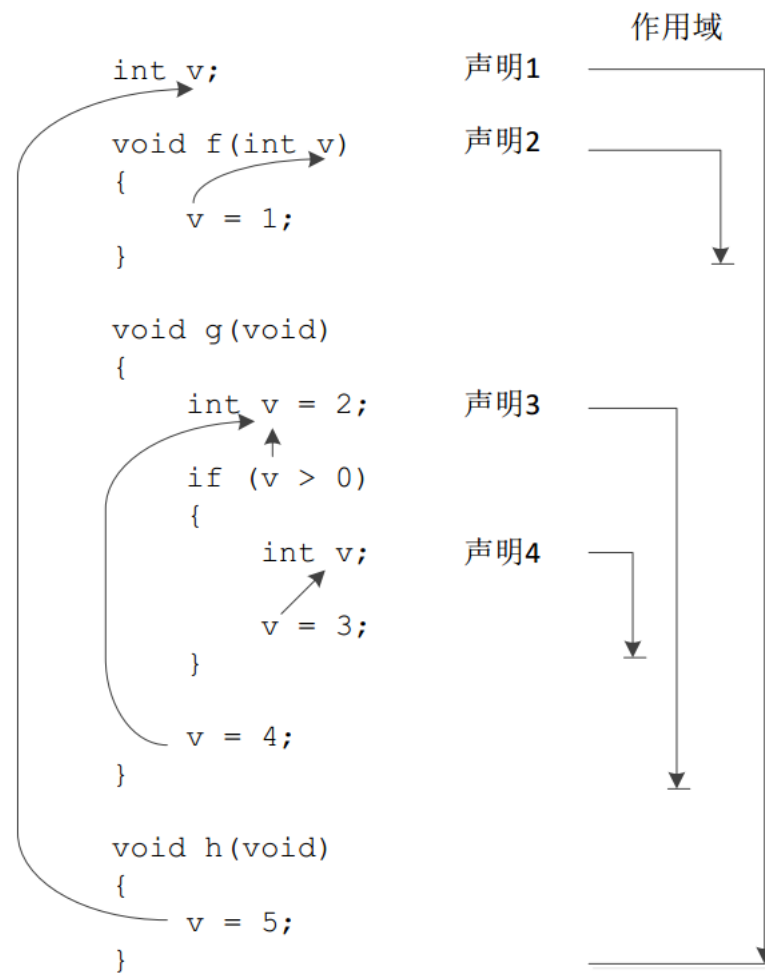
函数原型作用范围

✿ 函数原型中的参数。

```
int max(int x, int y);  
  
void main() {  
    ...  
}  
  
int max(int x, int y) {  
    ...  
}
```


作用范围

作用域存在重叠区的同名变量



关于声明和定义

定义声明

- ✿ 定义一个标识符，同时也是声明。
- ✿ 对于变量，定义意味着创建，即为其分配内存。
- ✿ 对于函数，需要定义函数体。

引用声明

- ✿ 只是声明一个需要引用的标识符。
- ✿ 例如
 - 函数的原型
 - 变量的引用声明需要带**extern**

案例分析：职工年龄分析程序

问题

- ✿ 定义一个数组，用来保存某单位全部**20**名职工的年龄信息；
- ✿ 定义函数**GetData**，从用户输入中读取年龄数据；
- ✿ 定义函数**Analysis**，对年龄数据进行分析，得到平均年龄以及年轻人（**<30**岁）、中年人（**30~50**岁）、老年人（**>50**岁）等各年龄段的人数；
- ✿ 定义函数**PrintResult**，显示分析结果；
- ✿ 定义主函数调用前面三个函数。

案例分析：职工年龄分析程序

分析

- ✿ 如果把保存年龄数据的数组的定义放在主函数中，就必须在定义另外两个功能函数的时候把数组作为参数以实现年龄数据的传递。
- ✿ 另外，年龄分析函数必须把统计得到的**1**个平均年龄和**3**个年龄段的人数反向传递给主函数，但是一个函数只有一个返回值。那么，可以利用全局变量作为多个函数共用的存储区域来实现函数间的数据双向传递。

案例分析：职工年龄分析程序

代码

```
#include <stdio.h>
#include <conio.h>

#define EMPLOYEENUM 20

/* 定义外部数组保存年龄数据 */
int gEmployee[EMPLOYEENUM];

/* 定义全局变量gAverage, gYoung, gMiddle和gOld */
double gAverage;
int gYoung = 0;
int gMiddle = 0;
int gOld = 0;

/* 从用户的输入读取年龄数据 */
void GetData();
/* 对年龄数据进行分析 */
void Analysis();
/* 显示分析结果 */
void PrintResult();
```

案例分析：职工年龄分析程序

代码

```
int main(void)
{
    GetData();
    Analysis();
    PrintResult();

    printf("请按任意键继续...");
    getch();

    return 0;
}

void GetData()
{
    int i;

    printf("输入%d位职工的年龄: \n", EMPLOYEEENUM);
    for (i=0; i<EMPLOYEEENUM; i++)
        scanf("%d", &gEmployee[i]);
}
```

案例分析：职工年龄分析程序

代码

```
void Analysis()
{
    int i;
    int sum = 0;

    for(i=0; i<EMPLOYEENUM; i++)
    {
        if(gEmployee[i]>50) gOld++;
        else if(gEmployee[i]>=30) gMiddle++;
        else gYoung++;

        sum += gEmployee[i];
    }

    gAverage = (double)sum/EMPLOYEENUM;
}

void PrintResult()
{
    printf("年青人数量: %d\n中年人数量: %d\n老年人数量: %d\n",
           gYoung, gMiddle, gOld);
    printf("平均年龄: %.1f\n", gAverage);
}
```

存储类别

四种存储类别说明符

- ✿ auto
- ✿ register
- ✿ extern
- ✿ static

两种存储时期

- ✿ 自动存储时期
 - auto
 - register
- ✿ 静态存储时期
 - extern
 - static

自动存储类别

自动存储

- ✿ 程序执行到变量（的定义声明）所在的程序块内时创建它，退出时销毁。

- 不会被自动初始化。

- ✿ **auto**（自动变量）：局部变量的缺省类别。

- 程序块中定义的变量，函数的参数。

```
auto int a, b;
```

- ✿ **register**（寄存器变量）：建议编译器把变量放进CPU的寄存器。

- 只适用于自动变量。

```
register int count = 1;
```

静态存储类别

静态存储

✿ 程序开始执行时创建，在程序执行期间，变量一直存在。

➤ 自动被初始化，缺省初值：0 或 ‘\0’。

✿ **static**（静态局部变量）：可以用于局部变量。

➤ 退出程序块后依然存在，并保留值。

➤ 但不改变其作用范围，即只能在所在的程序块内被使用。

```
static int a, b;
```

✿ 全局变量

➤ **extern**（外部变量）：用于引用声明在“外部”定义的全局变量。

```
extern int total;
```

案例分析：存储类别

存储类别

✿ 读下面的程序，写出结果。（cw0901.c）

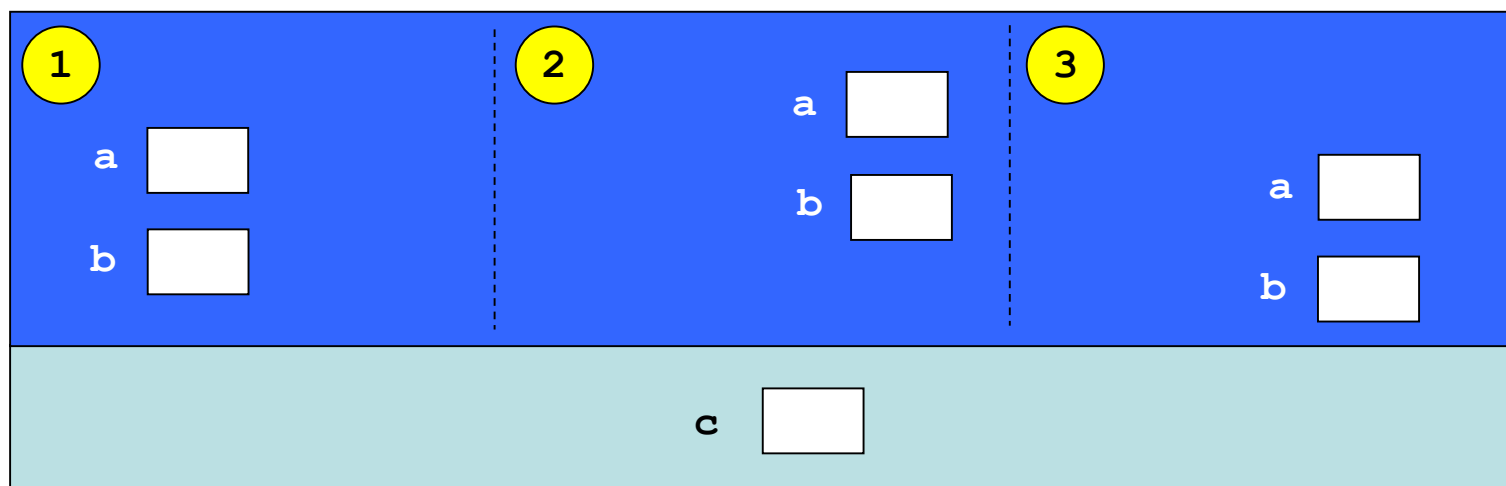
```
void try(int a) {  
    auto int b=10;  
    static int c=10;  
    b++;    c++;  
    printf("a=%d\tb=%d\tc=%d\n", a, b, c);  
}  
  
void main() {  
    int i;  
    for (i=1;i<=3;i++)  
        try(i);  
}
```

a=1	b=11	c=11
a=2	b=11	c=12
a=3	b=11	c=13

案例分析：存储类别

存储类别

结果分析

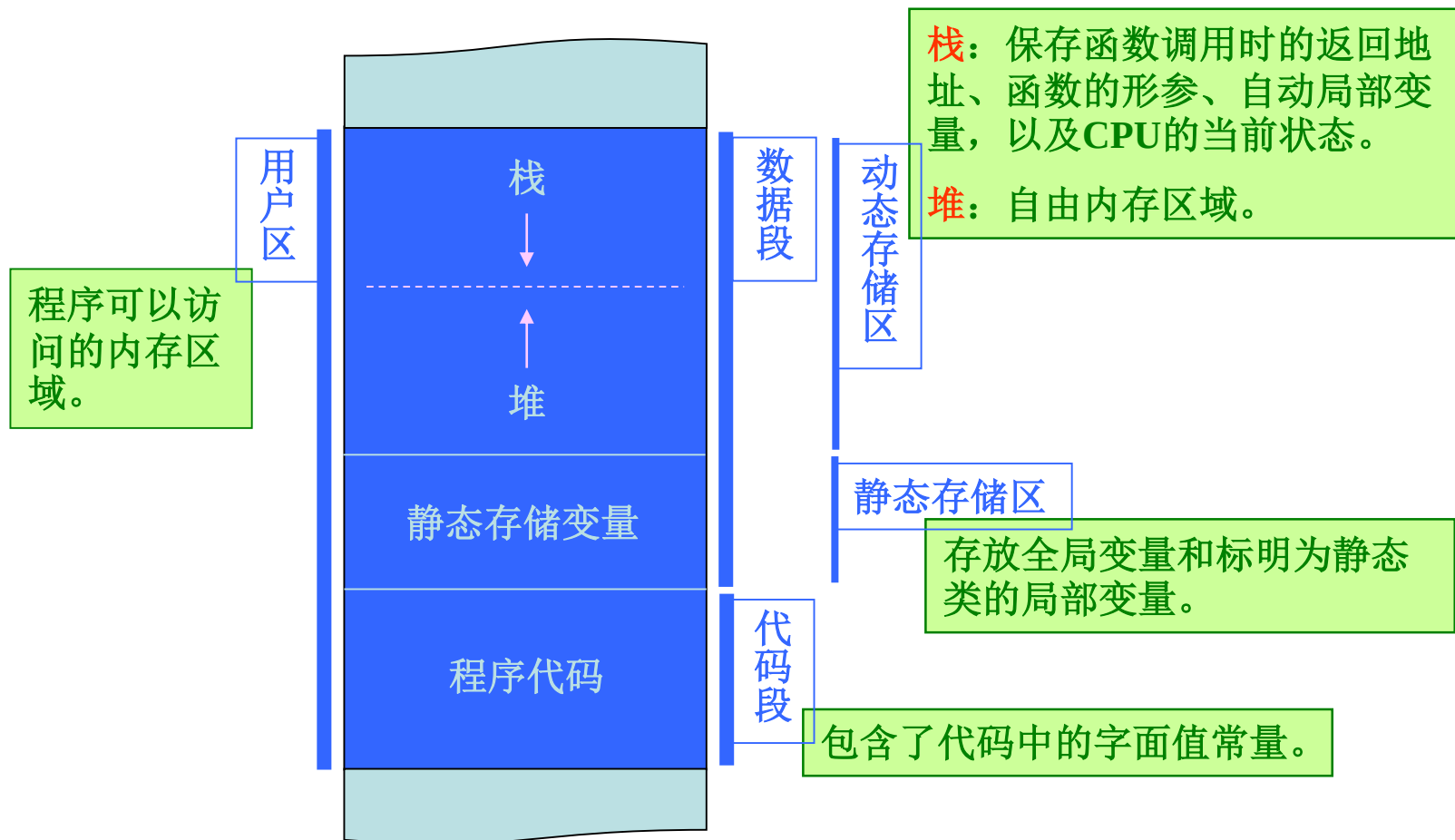


 动态存储区
 静态存储区

<code>a=1</code>	<code>b=11</code>	<code>c=11</code>
<code>a=2</code>	<code>b=11</code>	<code>c=12</code>
<code>a=3</code>	<code>b=11</code>	<code>c=13</code>

C语言程序的内存映像

C语言程序的内存映像



案例分析：外部变量

外部变量 (cw0902.c)

```
#include <stdio.h>
```

```
void main() {
```

```
    extern int a;
```

```
    int c=3;
```

```
    printf("a = %d\n", a);
```

```
    {
```

```
        extern int c;
```

```
        printf("inner c = %d\n", c);
```

```
    }
```

```
    printf("c = %d\n", c);
```

```
}
```

```
int a=1;
```

```
int c=5;
```

a = 1

inner c = 5

c = 3

案例分析：作用范围与存储类别

作用范围与存储类别 (cw0903.c)

```
#include <stdio.h>

void a(void); /*function prototype*/
void b(void); /*function prototype*/
void c(void); /*function prototype*/

int x = 1; /*global variable*/

void main() {
    int x = 5;
    printf("local x in outer scope of main is %d\n", x);

    { /*start new scope*/
        int x = 7;
        printf("local x in inner scope of main is %d\n", x);
    } /*end new scope*/

    printf("\nlocal x in outer scope of main is %d\n", x);
}
```

案例分析：作用范围与存储类别

作用范围与存储类别

```
a();  
b();  
c();  
a();  
b();  
c();  
  
printf("local x in main is %d\n", x);  
}  
  
void a() {  
    int x = 25; /*initialized each time a is called*/  
    printf("\nlocal x in a is %d after entering\n", x);  
    x++;  
    printf("local x in a is %d before exiting\n", x);  
}
```


案例分析：作用范围与存储类别

作用范围与存储类别

```
void b() {  
    static int x = 50; /*static initialization only*/  
                        /*first time b is called*/  
    printf("\nlocal x in b is %d after entering\n", x);  
    x++;  
    printf("local x in b is %d before exiting\n", x);  
}  
  
void c() {  
    printf("\nglobal x is %d on entering c\n", x);  
    x*=10;  
    printf("global x is %d on exiting c\n", x);  
}
```

案例分析：作用范围与存储类别

作用范围与存储类别

```
local x in outer scope of main is 5  
local x in inner scope of main is 7  
local x in outer scope of main is 5
```

```
local x in a is 25 after entering a  
local x in a is 26 before exiting a
```

```
local x in b is 50 after entering b  
local x in b is 51 before exiting b
```

```
global x is 1 on entering c  
global x is 10 on exiting c
```

案例分析：作用范围与存储类别

作用范围与存储类别

```
local x in a is 25 after entering a
local x in a is 26 before exiting a

local x in b is 51 after entering b
local x in b is 52 before exiting b

global x is 10 on entering c
global x is 100 on exiting c

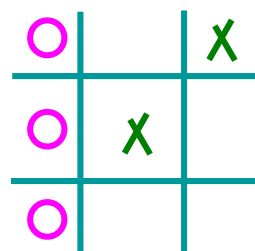
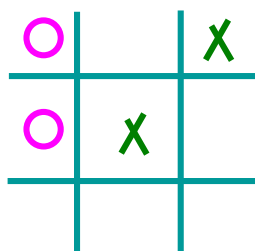
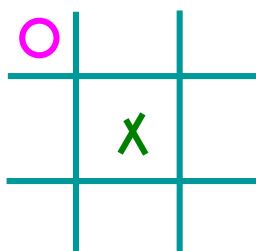
local x in main is 5
```

案例分析：TicTacToe

问题：“井”字棋游戏（一担挑）

游戏规则：

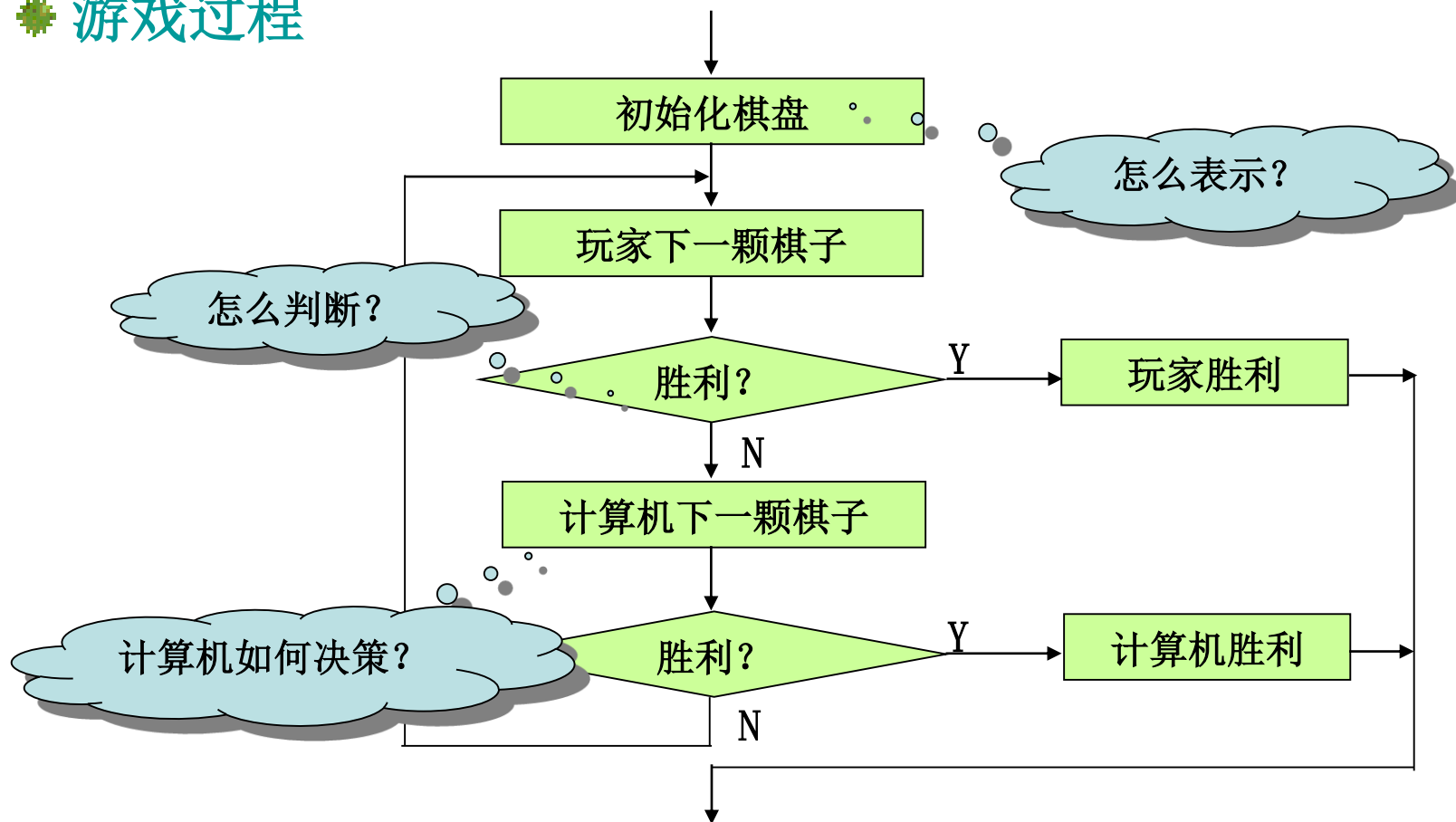
- 双方轮流下子
- 每次一方只能在某一个空格处下一颗棋子
- 胜负判断：
 - 若棋盘的某一行，或某一列，抑或某一对角线上的三个格子被某一方的棋子占据，则该方胜利；
 - 否则，为平局



案例分析：TicTacToe

分析

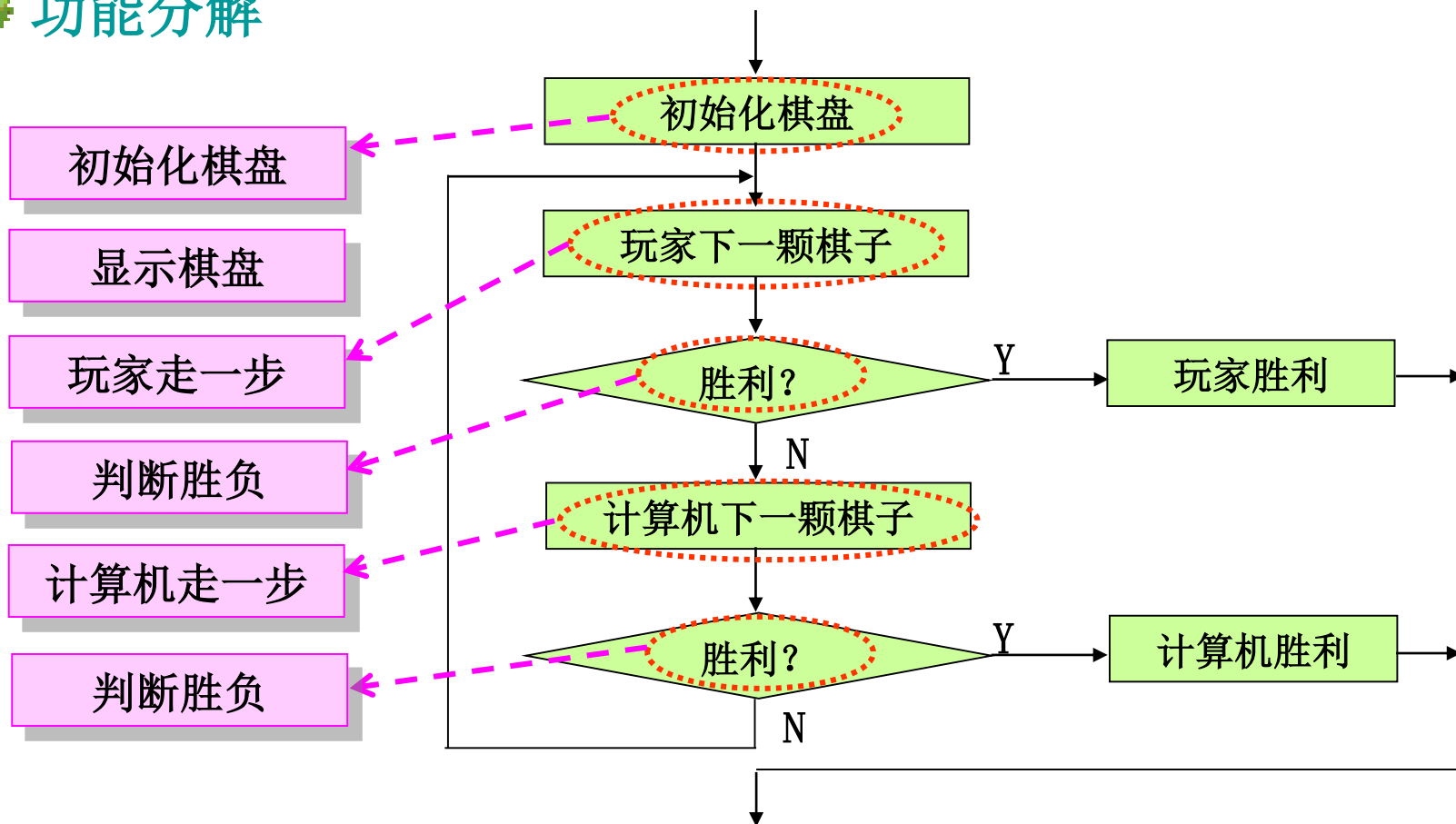
游戏过程



案例分析：TicTacToe

设计

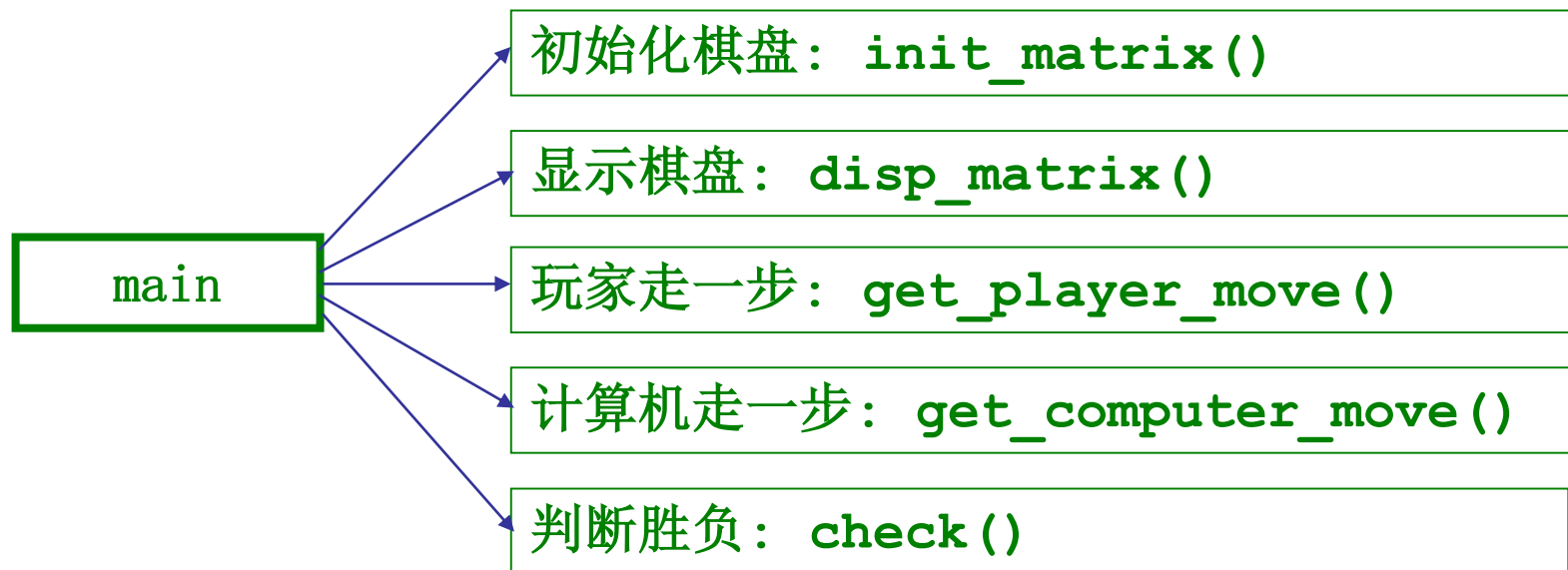
功能分解



案例分析：TicTacToe

设计

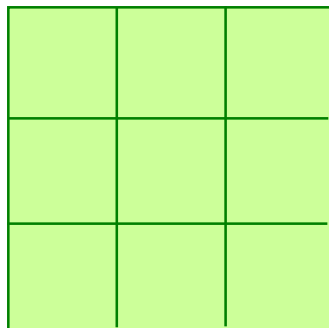
功能模块



案例分析：TicTacToe

设计

数据结构



3*3的二维字符数组：

```
char matrix[3][3];
```

全局变量。

- 玩家的棋子用 ‘X’表示；
- 计算机的棋子用 ‘O’表示；
- ‘ ’ 表示空。

一步棋

确定棋子的位置：坐标 (x, y)

案例分析：TicTacToe

实现 (cw1107.c)

源代码分析

```
char matrix[3][3]; /* the ticktacktoe matrix */

void main(void) {
    char done;

    printf("This is the game of ticktacktoe.\n");
    printf("You will be playing against the computer.\n");

    done = ' ';
    init_matrix();
}
```

done: 保存当前游戏的状态和结果

‘ ’ 游戏未结束，继续玩

‘x’ 游戏结束，玩家胜

‘o’ 游戏结束，计算机胜

案例分析：TicTacToe

实现

源代码分析

```
do {
    disp_matrix();
    get_player_move();
    done = check(); /* see if winner */
    if (done != ' ') break; /* winner! */
    get_computer_move();
    done = check(); /* see if winner */
} while (done == ' ');

if (done == 'X') printf("You won!\n");
else printf("I won!!!\n");
disp_matrix(); /* show final positions */
}
```

案例分析：TicTacToe

实现

源代码分析

```
/* Initialize the matrix. */  
void init_matrix(void) {  
    int i, j;  
  
    for (i=0; i<3; i++)  
        for (j=0; j<3; j++)  
            matrix[i][j] = ' ';  
}
```

案例分析：TicTacToe

实现

源代码分析

```
/* Display the matrix on the screen. */
void disp_matrix(void) {
    int t;

    for (t=0; t<3; t++) {
        printf(" %c | %c | %c",
               matrix[t][0], matrix[t][1], matrix[t][2]);
        if (t != 2) printf("\n---|---|---\n");
    }
    printf("\n");
}
```

案例分析：TicTacToe

实现

源代码分析

```
/* See if there is a winner. */
char check(void) {
    int i;

    for (i=0; i<3; i++) /* check rows */
        if (matrix[i][0] == matrix[i][1] &&
            matrix[i][0] == matrix[i][2])
            return matrix[i][0];

    for (i=0; i<3; i++) /* check columns */
        if (matrix[0][i] == matrix[1][i] &&
            matrix[0][i] == matrix[2][i])
            return matrix[0][i];
}
```

案例分析：TicTacToe

实现

源代码分析

```
/* test disgonals */  
if (matrix[0][0] == matrix[1][1] &&  
    matrix[1][1] == matrix[2][2])  
    return matrix[0][0];  
  
if (matrix[0][2] == matrix[1][1] &&  
    matrix[1][1] == matrix[2][0])  
    return matrix[0][2];  
  
return ' ';
```

```
}
```

案例分析：TicTacToe

实现

源代码分析

```
/* Get a player's move. */
void get_player_move(void) {
    int x, y;

    printf("Enter X, Y coordinates for your move: ");
    scanf("%d%c%d", &x, &y);

    x--; y--;

    if (matrix[x][y] != ' ') {
        printf("Invalid move, try again.\n");
        get_player_move();
    }
    else matrix[x][y] = 'X';
}
```

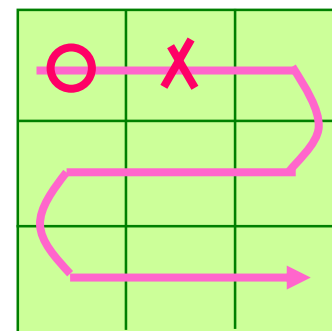
案例分析：TicTacToe

实现

```
/* Get a move from the computer. */
void get_computer_move(void) {
    int i, j;

    for (i=0; i<3; i++) {
        for (j=0; j<3; j++)
            if (matrix[i][j] == ' ') break;
        if (matrix[i][j] == ' ') break;
    }

    if (i*j == 9) {
        printf("draw\n");
        exit(0);
    }
    else
        matrix[i][j] = 'O';
}
```



☺人工智能

包含多个源文件的程序

包含多个源文件的程序

- 每个函数的定义必须在一个文件内，不能被分割。
- 全局变量可以被同一文件内的函数访问。
 - 如果需要被其他文件内的函数访问，则必须在其他文件内声明。



➤ **extern** 表示变量 `myGlobal` 是在另一个文件内有定义。

- 一个文件内定义的函数，也可以被其他文件内的函数调用。
 - 在每个文件内加入该函数的原型（声明为外部函数）。
 - 函数的声明可以不需要 **extern**。

包含多个源文件的程序

包含多个源文件的程序

举例

```
int a, b;  
extern int max(void);  
void main() {  
    scanf("%d%d", &a, &b);  
    printf("%d", max());  
}
```

cw0904a.c

A

```
extern int a, b;  
  
int max() {  
    return (a>b?a:b);  
}
```

cw0904b.c

B

包含多个源文件的程序

包含多个源文件的程序

✿ static

- 限制全局变量只能被同一文件内的函数访问。
- 限制函数只能被同一文件内的函数调用。
- 说明：“同一文件”是指包含变量的定义、函数的定义的那个文件。

```
static int myGlobal;  
  
static void myFunc() {  
    ...  
}
```

编译包含多个源文件的程序

编译包含多个源文件的程序

- ✿ 每个源文件必须被编译，然后链接成一个可执行文件。
- ✿ 如果有一个文件作了改动，则必须重新编译所有相关的文件。
- ✿ 一般会提供 **make** 工具用来管理和编译多源文件的程序。
 - 创建 **makefile** 文件，记录编译规则。
 - 自动查找必须编译的源文件。
- ✿ 可以创建**项目** (**project**) 文件来管理多源文件的程序。
 - 举例：演示在**VC**中创建工程，并加入多个文件。

VC2010 中的多源文件程序

VC2010的文件管理

✿ 解决方案 (solution)

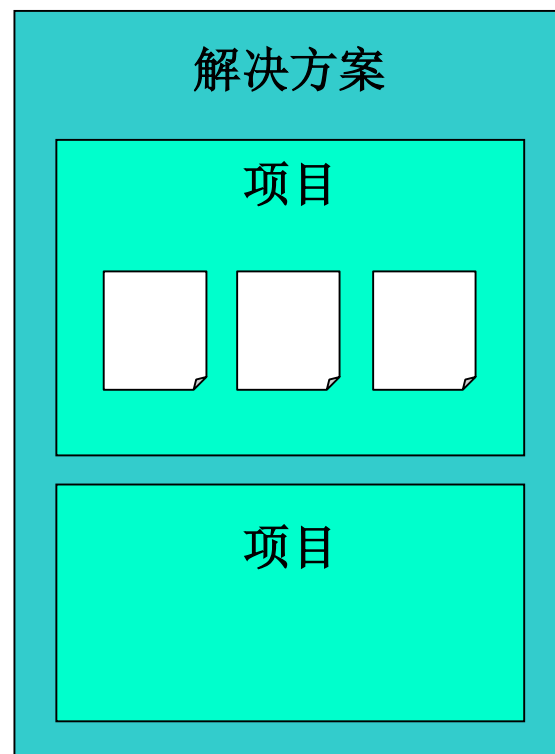
- .sln
- 可以包含多个项目

✿ 项目 (project)

- .vcxproj
- 一个项目就是一个程序
- 可以包含多个文件

✿ 文件 (file)

- 源文件 (.c)
- 头文件 (.h)
-



小结

- 用于存储程序数据的内存可用存储时期和作用范围来表征。
- 存储时期可以是静态的或自动的。
 - ✿ 如果是静态的，变量需要使用的内存在程序开始执行时被分配，并在程序执行时一直占用（存在）。
 - ✿ 如果是自动的，变量需要使用的内存在程序执行到该变量定义所在的代码块时开始分配，在退出代码块时释放。
- 作用范围决定了哪一部分程序可以访问某个对象。
- C语言程序可以包含多个源文件，在编译时，逐一编译各文件，然后通过链接器链接所有的目标代码构成可执行文件。